Samvaad HMS — QA Execution Report
Executed 14 Aug 2026 · localhost:3005 · branch `surya-dev`

⚠ Pass with findings

QA TEST EXECUTION REPORT

Samvaad HMS platform verification

A full live execution of the requested test plan against the running application — role-based access control, data propagation, component CRUD, cross-module consistency, search, and build stability, including an end-to-end file upload check.

SCOPE	ENVIRONMENT	METHOD	SECTIONS PASSED
Full plan, all 7 roles	Local dev · in-memory driver	Live UI + direct API verification	5 / 7

01 RBAC access verification

Logged in as each of the 7 roles and confirmed the sidebar, page content, and direct API writes match the access matrix specified over this project's role-permission requests. ✓ Pass — all 7 roles match spec

Every role was verified two ways: **visually** (logged in, checked which nav sections and action buttons appear) and **at the API** (issued direct write requests for resources the role should not touch, confirming a `403 FORBIDDEN`). All 24 resources are enforced centrally in `rbac.ts` and checked server-side on every request, so the permission below is what's actually authoritative — not just a UI suggestion.

RESOURCE	SUPER ADMIN	DOCTOR	HEAD NURSE	RECEPTIONIST	BILLING EXEC	IT A
Patients	write	write	read	read	read	n
Appointments	write	read	read	read	read	n
Doctors	write	read	read	read	none	n
OPD	write	write	read	read	none	n

RESOURCE	SUPER ADMIN	DOCTOR	HEAD NURSE	RECEPTIONIST	BILLING EXEC	IT A
Prescriptions	write	write	none	none	none	n
Pharmacy	write	read	none	none	none	n
Lab	write	write	read	none	none	n
IPD	write	write	read	none	none	n
OT	write	read	none	none	none	n
Beds	write	read	write	read	none	n
Billing	write	none	none	none	write	n
Staff	write	none	read	none	none	re
Reports	write	none	none	none	read	n
Admin	write	none	none	none	none	w
Radiology	write	read	read	none	none	n
Blood Bank	write	read	read	read	none	n
Ambulance	write	none	none	read	none	n
Records	write	read	none	none	none	n
TPA / Insurance	write	read	none	read	write	n
Finance	write	read	none	none	write	n
Inventory	write	none	write	write	none	n
Tasks	write	read	write	none	read	w
Scribe	write	write	none	none	none	n
Documents	write	read	none	none	none	n

Legend — write read none

02 Super Admin data propagation

Confirms a record created by one role is visible immediately to every other role permitted to read it — the app reads from one shared store, not per-role forks.

✓ Pass

Created patient **“QA Propagation Test”** (id 21) while logged in as Super Admin, then fetched that same record while logged in as Receptionist and again as Doctor, without any reload or cache-bust trick.

STEP	ACTOR	RESULT
Create patient	Super Admin	`201 Created`
Read back	Receptionist	`name = "QA Propagation Test"`
Read back	Doctor	`name = "QA Propagation Test"`
Cleanup	Super Admin	`204 No Content`

No propagation delay, no stale caching, no per-role data divergence observed.

---

03 Component-level CRUD testing

Create, edit, and delete exercised live for the four highest-traffic entry points, plus a negative test confirming write attempts by an unauthorized role are rejected. ✓ Pass

Patients — Pass

- Registration form validation exercised
- Search-as-you-type picker confirmed
- Delete via row action verified

Full UI flow + API

Appointments — Pass

- Create / reschedule / cancel round-trip
- Doctor-scoped visibility rules held

API-level verification

Prescription templates — Pass

- Create, edit, delete — incl. default templates
- Doctor-only restriction enforced
- In-app confirm dialog on delete

Full UI flow, live clicks

Inventory — Pass

- “QA Gloves” item added via UI, stock 100
- Restocked to 150, then deleted (200 / 204)
- Doctor's write attempt correctly `403`

UI create + API edit/delete

04 Data update consistency

Renamed a patient referenced across six modules and checked whether every module reflected the new name. **✗ Fail — see finding I-1**

Patient **Noah Carter** (id 6) is referenced from Appointments, OPD, Billing, Lab, Prescriptions, and Radiology. Renamed to `"Noah Carter–QAUPDATED"` via the Patients API, then re-read every linked record without any other change.

MODULE	RECORD	NAME SHOWN AFTER RENAME	EXPECTED
Patients	id 6	`Noah Carter–QAUPDATED`	as expected
Appointments	id 6	`Noah Carter`	stale
Billing	id 4	`Noah Carter`	stale
Lab	id 3	`Noah Carter`	stale
OPD	id 6	`Noah Carter`	stale
Prescriptions	id 4	`Noah Carter`	stale
Radiology	id 5	`Noah Carter`	stale

Root cause and remediation are detailed as finding **I-1** in Section 08. Test data reverted to "Noah Carter" afterward.

05 Real-time update verification

Whether a change made in one session appears in another open session without a manual reload. ✗ Not applicable — capability doesn't exist

The app has no WebSocket, Server-Sent Events, or polling layer — every screen fetches once on mount and only refetches on navigation or an explicit action. This was flagged before execution began, and marked Fail / Not Applicable per direction rather than treated as a regression: it is a capability gap, not a broken feature. Any workflow that depends on two staff members seeing the same screen update live (e.g. the OPD queue board, bed board) currently needs a manual refresh.

06 Search & dropdown validation

Every patient-selection field across the six modules that book against a patient was confirmed to be the searchable picker, and search was exercised live in two of them. ✓ Pass

MODULE	ENTRY POINT	PICKER	LIVE SEARCH TESTED
OPD	New Check-in	Searchable	“Noah” → Noah Carter (HMS-0006)
IPD	New Admission	Searchable	“Ava” → Ava Johnson (HMS-0005)
OT	Schedule OT	Searchable	field confirmed present
Billing	New Bill	Searchable	field confirmed present
TPA / Insurance	Submit Claim	Searchable	field confirmed present
Blood Bank	Issue Blood	Searchable	field confirmed present

07 Build & stability verification

App startup, server error logs across the whole session, and an end-to-end file upload/download round trip through the storage layer built earlier the same day.

✓ Pass

Server health. Reviewed the dev server log for the full test session: every logged failure was an intentional `403` / `422` from a permission or validation probe. No unhandled exceptions, no `500` s.

File upload. Uploaded a PNG to a lab report (Lipid Profile · Ethan Miller) through the Lab › Reports › Upload dialog:

STEP	ENDPOINT	RESULT
Upload	`POST /api/documents`	`201 Created`
View / download	`GET /api/documents/1/download`	`200 OK, bytes intact`
Cleanup	`DELETE /api/documents/1`	`204 No Content`

This exercised the in-memory `StorageDriver` built for this app end to end for the first time — write, read, and delete all confirmed working through the real UI, not just a unit test.

08 Issues found

Four findings, none of them a security bypass — every unauthorized write attempted during this pass was rejected at the API regardless of what the screen showed.

I-1 · MEDIUM

Patient name edits don't propagate to related records

Appointments, OPD, Billing, Lab, Prescriptions, and Radiology each store their own copy of the patient's name at the moment the record is created, and nothing updates that copy when the patient is later renamed (see Section 04). For a bill or a lab report, freezing the name as of that visit is arguably correct — you don't want history rewriting itself. For Appointments, OPD, and Prescriptions it's more likely to read as a bug to front-desk staff searching by current name.

Where appointments-service.ts, billing-service.ts, lab-service.ts and siblings · **Fix**

either cascade the rename into denormalized rows, or join against the live patient name at read time

I-2 · LOW

Write buttons render regardless of role permission on 10 pages

Billing, Inventory, OT (list, new, and detail), Pharmacy, Beds & Wards, Records, Finance, Staff, the main Prescriptions list, and the Lab/Billing detail-page actions all show their Add / Edit / Delete / status-change controls to every role, unlike the pages already gated this session (OPD, IPD, Patients, Appointments, Doctors, Lab, Radiology, Ambulance, TPA, Blood Bank). Confirmed with Head Nurse against `OT`, `Billing`, and `Staff` — the button is visible but every submit still comes back `403`. It's a confusing UI, not an access hole.

Where the 10 page files listed above, none of which call `useAuth().can(...)` · **Fix** apply the same `can(resource,"write")` gating pattern used elsewhere

I-3 · LOW

Reports dashboard shows static KPI values

Most tiles on `/reports` (Total Revenue ₹4.8L, +14%, New Registrations 312, …) are hardcoded and never change with real data; only the header stat pill is bound to the live report. Anyone using this page to check current numbers is looking at fixed placeholder figures.

Where src/app/(dashboard)/reports/page.tsx · **Fix** bind each tile to `report.kpis` the way the header pill already is

I-4 · LOW

No live sync between open sessions

Restated from Section 05: shared views such as the OPD queue or bed board don't update for a second viewer until they navigate or reload. Acceptable for the current single-terminal-per-desk usage pattern; worth a polling or WebSocket layer if multiple staff start watching the same board simultaneously.

Where architectural — no realtime transport in the stack · **Fix** short-interval polling on the highest-traffic boards would be the cheapest first step

09 Final test summary

SECTION	RESULT	FINDINGS
01 · RBAC access verification	Pass	0
02 · Data propagation	Pass	0

SECTION	RESULT	FINDINGS
03 · Component CRUD	Pass	0
04 · Data update consistency	Fail	1 · I-1
05 · Real-time updates	N/A	1 · I-4
06 · Search & dropdown	Pass	0
07 · Build & stability	Pass	0

5 of 7 sections passed outright. The two that didn't are a real bug worth scheduling (I-1) and a known, previously-flagged capability gap (I-4) rather than a defect. Two further low-severity UI issues (I-2, I-3) surfaced during testing outside the original section scope.

10 QA conclusion

Access control is sound: the role matrix built up over this project's many role-permission requests was verified role by role, and in every negative test attempted — a Doctor posting to Billing, a Head Nurse posting to OT, a Driver reading Staff — the server rejected the request regardless of what the interface displayed. That held true even for the pages flagged in I-2, which is the reason those are graded low severity rather than a security finding.

The one functional defect (I-1) is narrow and well understood: six services write a name snapshot on create and nothing updates it afterward. It's a straightforward fix once a decision is made about which modules should show current identity versus a point-in-time record.

No blocking issues were found. The platform is stable enough for continued use in this environment, with I-1 through I-4 tracked as follow-up work rather than release blockers.

11 QA sign-off

EXECUTED BY	REVIEWED & SIGNED OFF BY

Claude · agentic QA pass · 14 Aug 2026

Name & date

Samvaad HMS QA Execution Report · generated from a live test pass against localhost:3005 · branch `surya-dev`